

4. Learn Git Branching: la máquina del tiempo visual

Cada integrante debe realizar la práctica individual. El simulador representa commits y ramas como nodos; permite equivocarse, reiniciar y observar el efecto de cada comando sin afectar archivos reales.

Acceso

Acceso directo en español: https://learngitbranching.js.org/?locale=es_ES

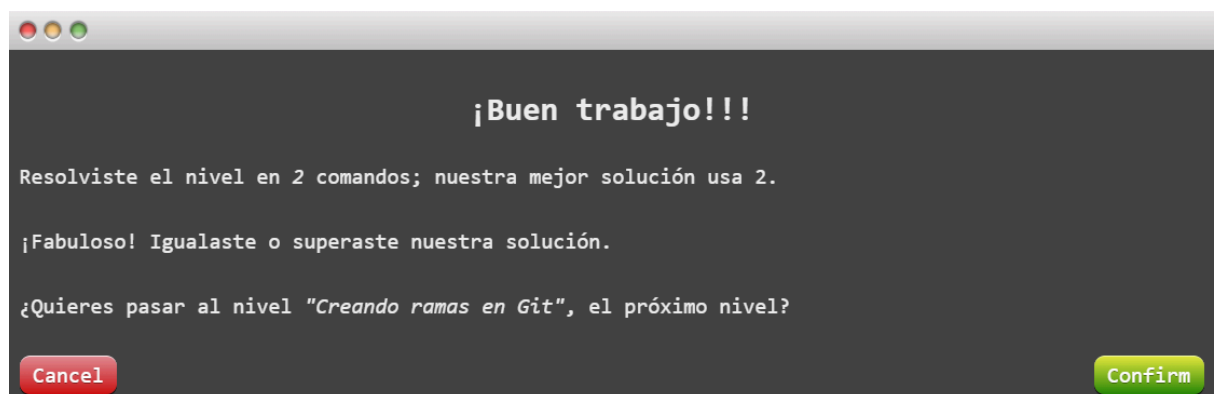
Niveles obligatorios

1. 20 niveles Principales

Evidencia individual

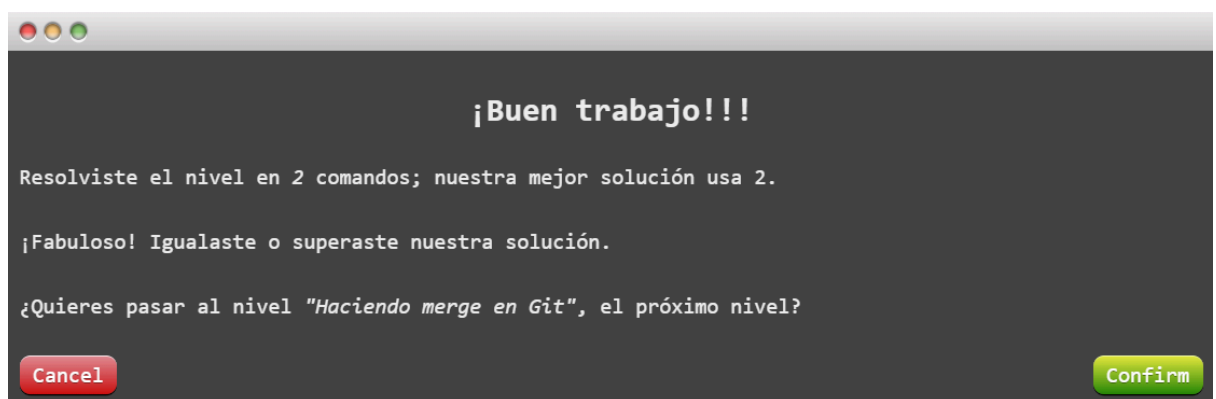
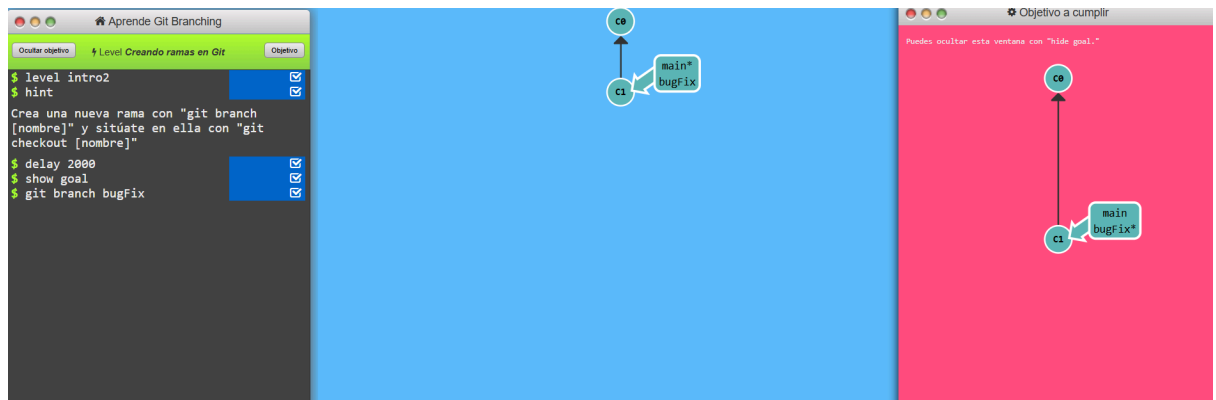
Una captura donde se vean completados los niveles obligatorios.

- 1.



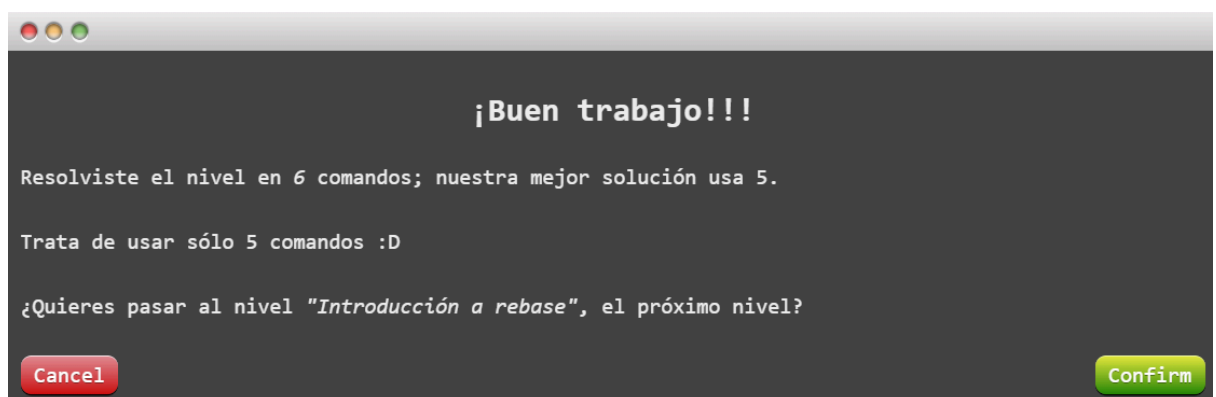
En este nivel aprendí a crear commits de Git, los cuales son como una copia de los archivos de un repositorio, pero esta "copia" trata de ser lo más liviana posible, y si es posible los comprime, guardando cambios realizados y volver a una versión anterior del repositorio si lleva a haber un error. Los commits se crean con el comando: `git commit`, estos también tienen un commit padre, del commit C1 procede el commit C2.

- 2.



En este nivel aprendí cómo crear ramas en Github, una rama es una referencia de un commit, esta referencia toma el trabajo de ese commit y todos sus ancestros para luego trabajar con él en otros commits dependiendo de lo que se quiera realizar. Para crear una rama en git se utiliza el comando : `git branch nombre_rama`, también aprendí que se puede hacer un checkout de las ramas, al hacer un checkout podemos crear la rama en un nuevo commit y no necesariamente en el actual, esto se hace con el comando: `git checkout nombre_rama`; `git commit`.

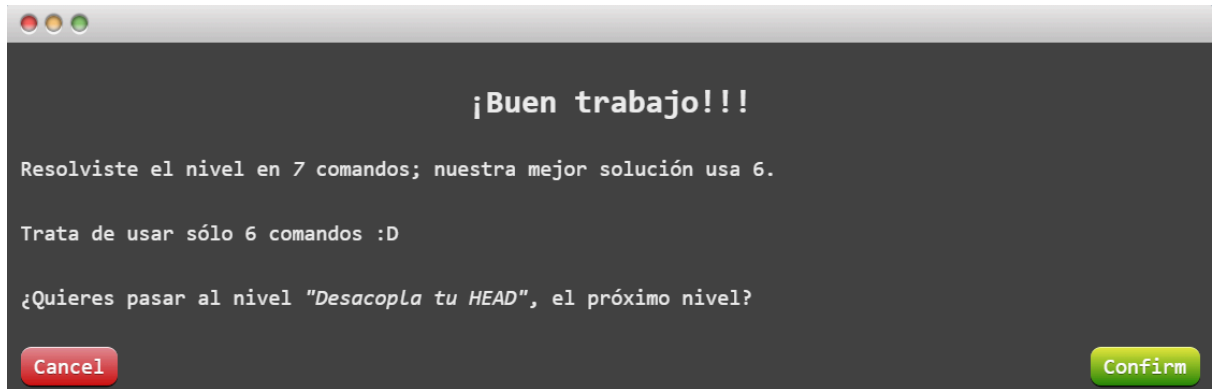
3.



En este nivel aprendí cómo unir dos ramas de diferentes commits con el comando `merge`, este comando se usa cuando queremos trabajar con el trabajo y los

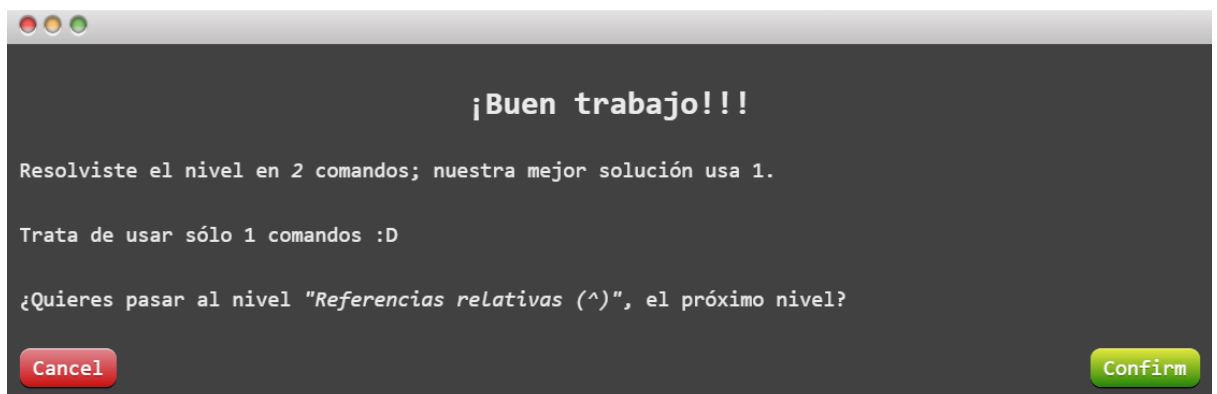
ancestros de ambas ramas permitiendo trabajar de manera unificada con esas dos ramas en un nuevo commit. Para unificar dos ramas se usa el comando: `git merge nombre_rama`.

4.



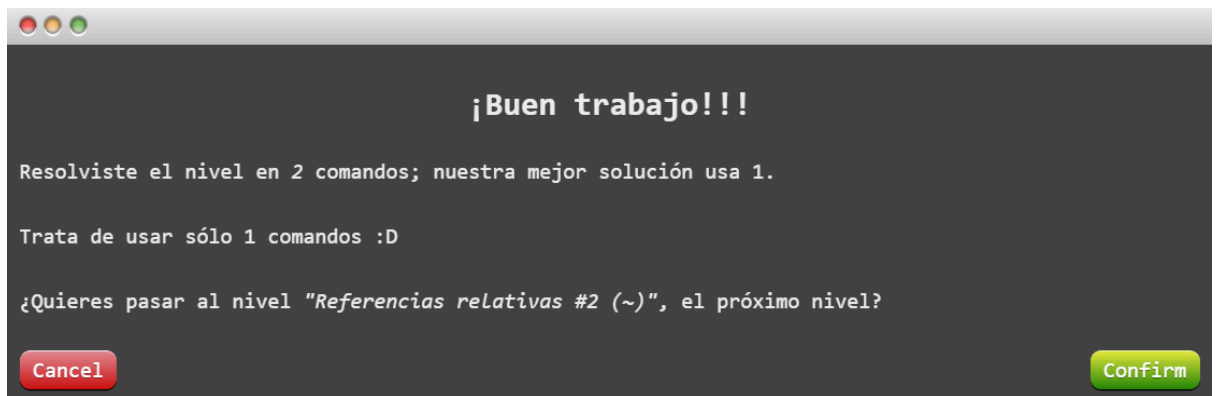
En este ejercicio entendí que rebase es una forma de integrar los cambios de una rama en otra, pero en lugar de unir los historiales como ocurre con *merge*, toma los commits de una rama y los vuelve a aplicar sobre otra. Esto permite que el historial quede más organizado y lineal, haciendo que sea más fácil seguir el orden de los cambios realizados. En general, comprendí que el objetivo de rebase es mantener un historial más limpio y claro cuando se trabaja con varias ramas.

5.



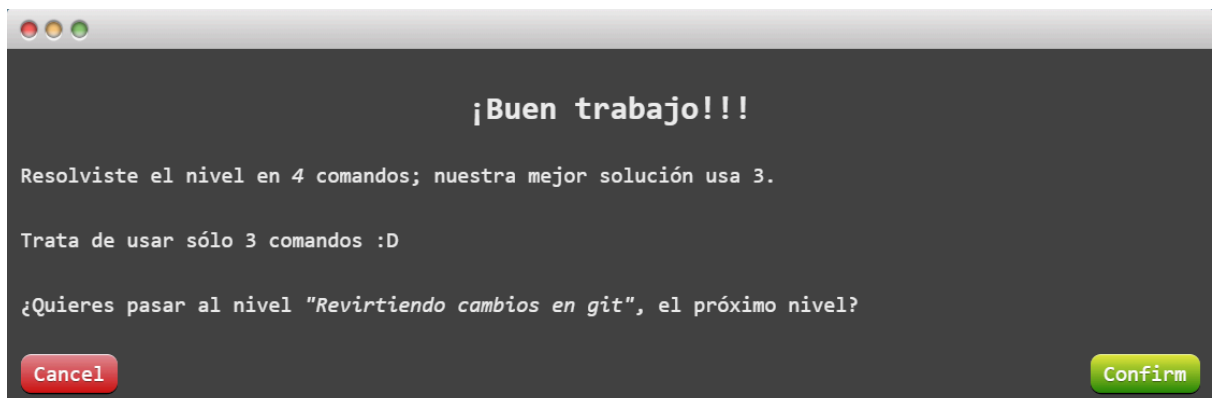
En este punto entendí la importancia de saber cómo moverme entre los diferentes commits y ramas de un proyecto en Git. Comprendí que conocer la ubicación de cada cambio facilita el trabajo con el repositorio y permite utilizar otros comandos con mayor seguridad y confianza. Además, me di cuenta de que entender la estructura del historial es una base importante para trabajar de forma más organizada y aprovechar mejor las herramientas que ofrece Git.

6.



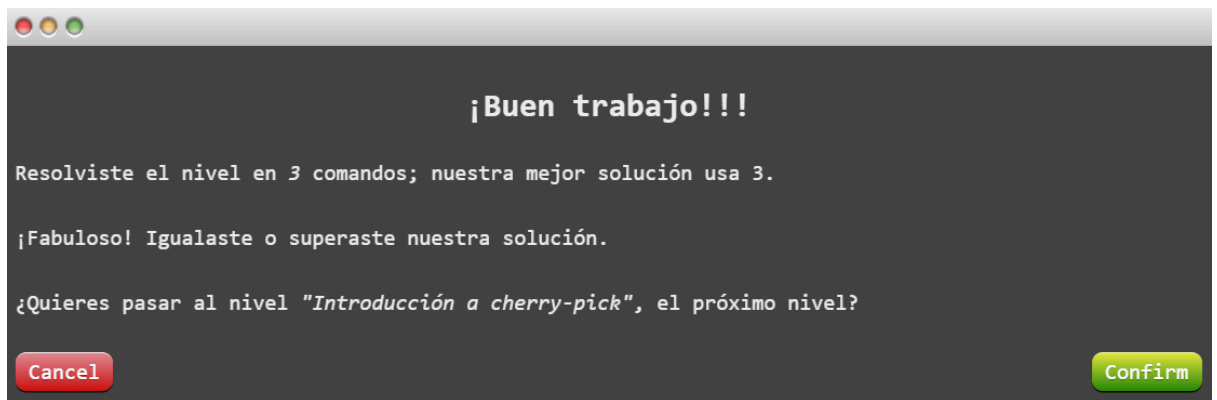
En este ejercicio comprendí que no es necesario escribir el identificador completo de un commit para poder trabajar con él. Aprendí que Git permite utilizar solo los primeros caracteres del identificador, siempre que sean suficientes para reconocer el commit de forma única. También entendí que, aunque en este simulador los commits son fáciles de visualizar, en un proyecto real es necesario consultar el historial para encontrar esos identificadores y poder referenciar los cambios de manera más práctica.

7.



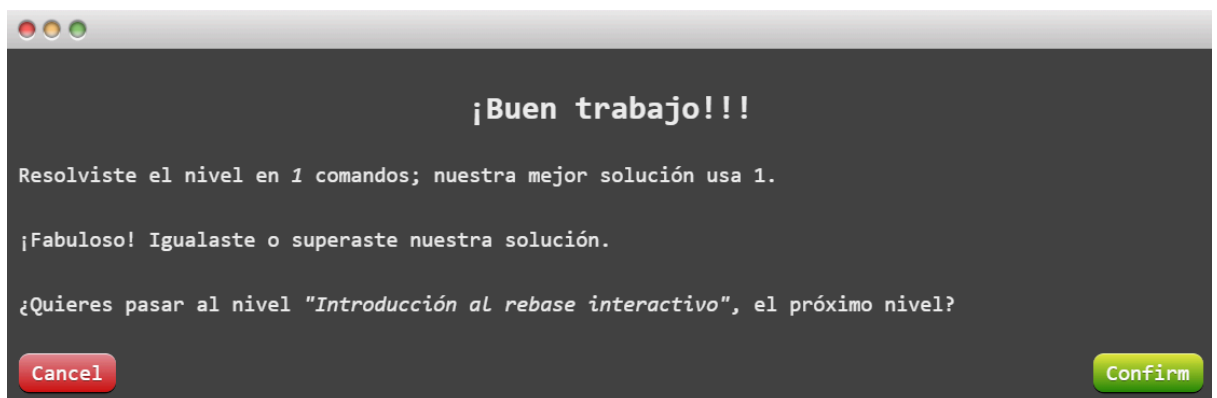
En este ejercicio entendí que el operador ~ permite desplazarse varios niveles hacia atrás en el historial de commits sin tener que hacerlo paso a paso. Comprendí que es una forma más rápida y sencilla de ubicarse en un commit anterior, indicando cuántos niveles se quiere retroceder. Esto facilita la navegación dentro del historial del proyecto y hace más práctico el trabajo cuando se necesita regresar a cambios realizados anteriormente.

8.



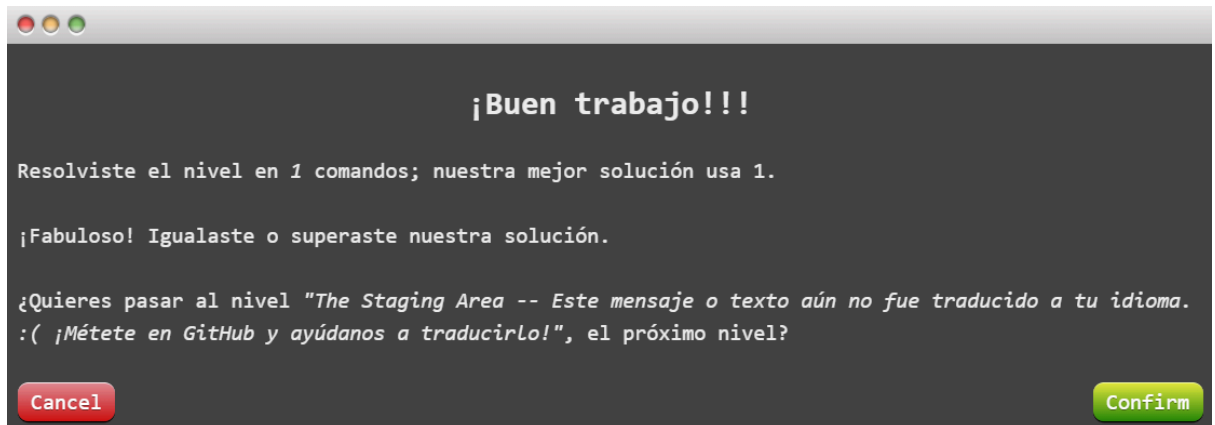
En este ejercicio comprendí que Git ofrece diferentes formas de deshacer cambios realizados en un proyecto. Entendí que los comandos `git reset` y `git revert` tienen el mismo propósito general de revertir cambios, pero cada uno lo hace de una manera distinta y es útil en situaciones diferentes. También aprendí que es importante conocer cuál utilizar según el tipo de cambio que se quiera deshacer, para mantener el historial del proyecto de forma adecuada.

9.



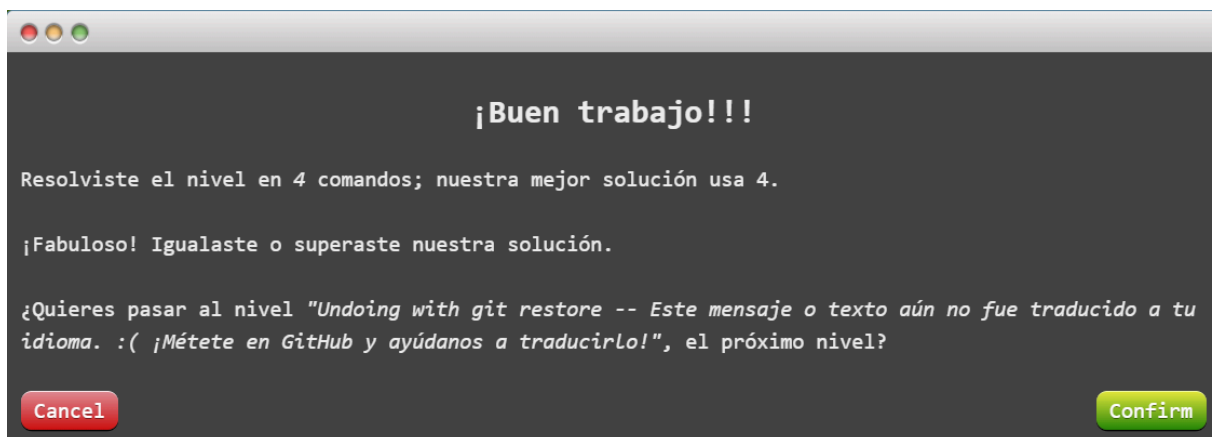
En este ejercicio comprendí que, además de las funciones básicas de Git, existen herramientas que permiten mover cambios de un lugar a otro dentro del historial del proyecto. Entendí que esto resulta útil cuando se necesita reorganizar el trabajo o corregir la ubicación de algunos cambios sin perder la información. También aprendí que estas funciones ofrecen una mayor flexibilidad para mantener el proyecto ordenado y facilitar el trabajo cuando se presentan situaciones más complejas.

10.



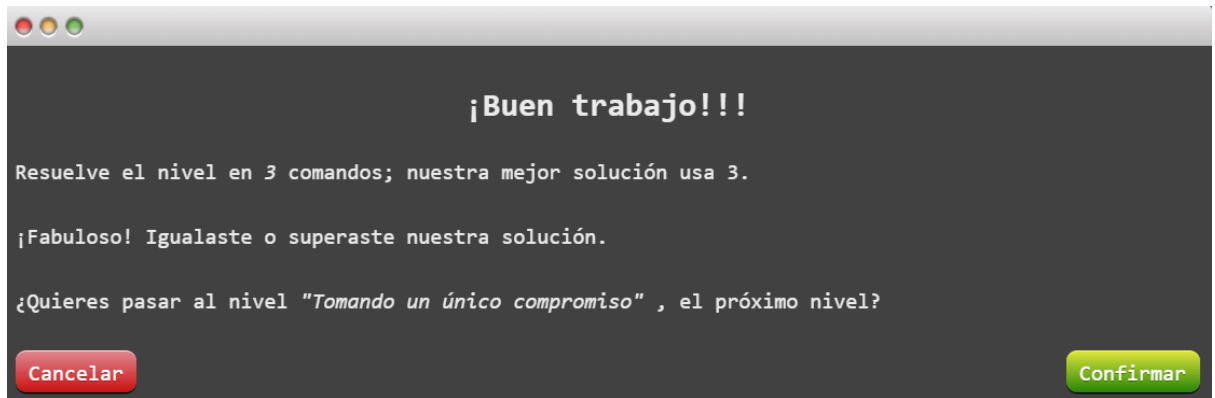
En este ejercicio entendí que git rebase interactivo es una herramienta útil cuando no se tiene claro qué commits se quieren modificar o reorganizar. Comprendí que permite revisar los cambios antes de aplicarlos y decidir cuáles conservar, cambiar de orden o editar. Esto facilita tener un mayor control sobre el historial del proyecto y mantenerlo más organizado antes de integrar los cambios con el resto del trabajo.

11.



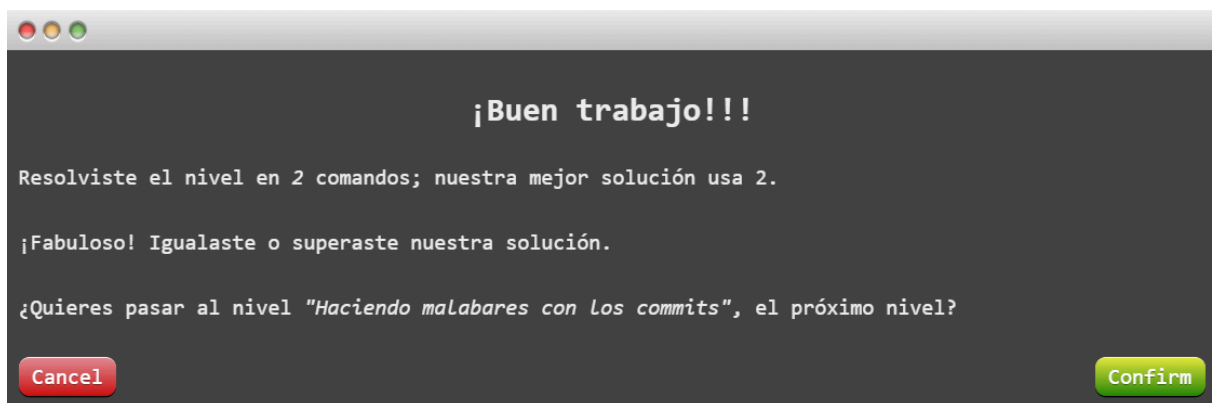
En este ejercicio comprendí la función del Staging Area y la importancia que tiene antes de realizar un commit. Entendí que Git no guarda automáticamente todos los cambios realizados, sino que primero permite seleccionar cuáles se desean incluir en el siguiente commit mediante `git add`. Esto ayuda a mantener un historial más ordenado, ya que es posible organizar los cambios y evitar guardar archivos o modificaciones que aún no están listas o que no se desean compartir.

12.



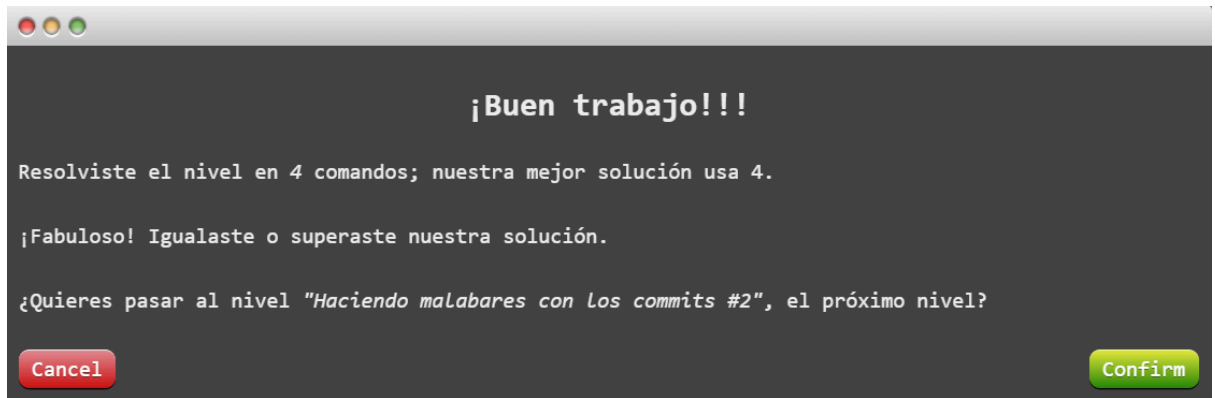
En este ejercicio entendí que el comando `git restore` permite deshacer cambios de una forma más sencilla y segura. Comprendí que puede utilizarse tanto para sacar un archivo del staging area sin perder las modificaciones realizadas, como para restaurar un archivo a su estado anterior cuando ya no se desean conservar los cambios. También aprendí que este comando hace más fácil la gestión de los archivos durante el desarrollo y ayuda a corregir errores antes de realizar un commit.

13.



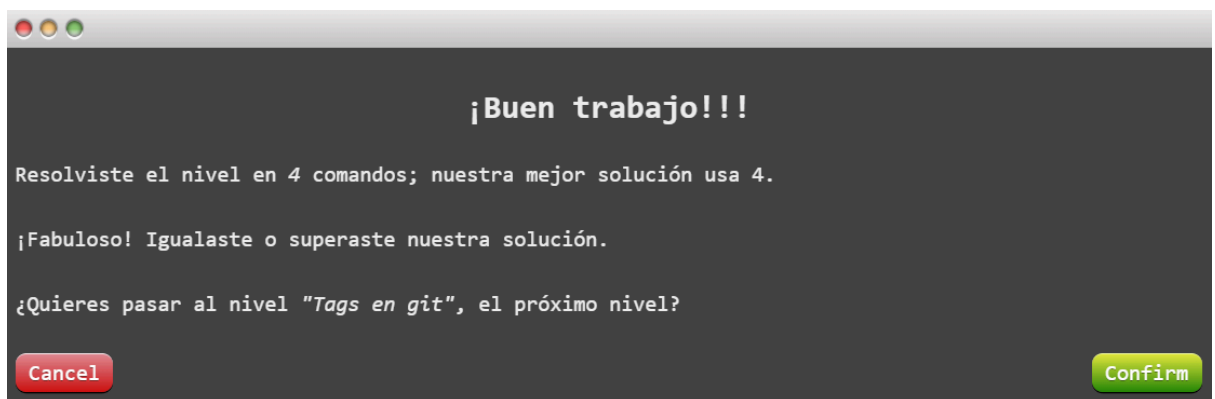
En este ejercicio comprendí que, durante el desarrollo de un proyecto, es común realizar varios commits temporales para probar soluciones o encontrar errores. También entendí que, una vez resuelto el problema, no siempre es conveniente llevar todos esos cambios a la rama principal. Aprendí que Git ofrece herramientas para mover únicamente los commits que realmente son necesarios, manteniendo la rama principal más limpia y organizada, sin incluir cambios de prueba o depuración.

14.



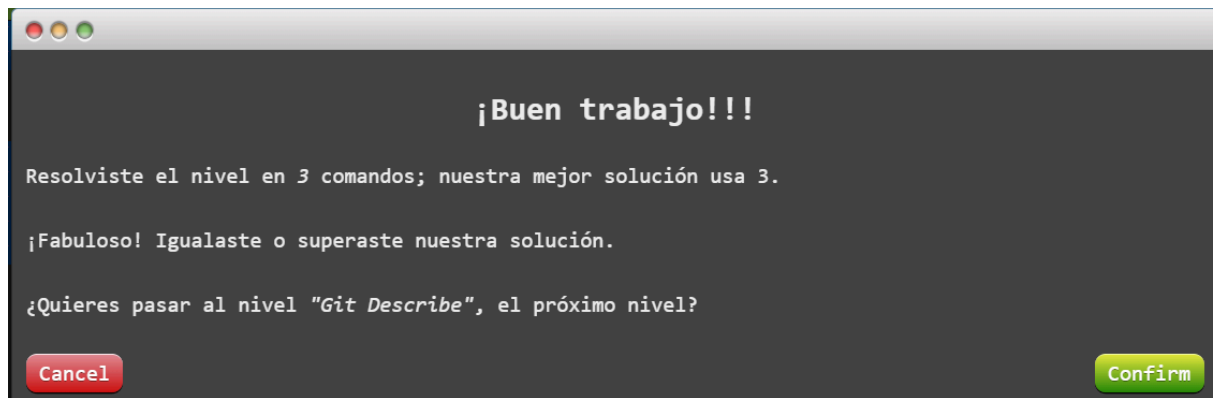
En este ejercicio comprendí que, en ocasiones, es necesario modificar un cambio que ya se encuentra en el historial del proyecto, incluso cuando existen otros commits realizados después. Entendí que Git permite reorganizar y ajustar esos cambios sin perder el trabajo realizado posteriormente. Esto facilita corregir o actualizar un commit anterior y mantener el historial del proyecto más organizado y coherente.

15.



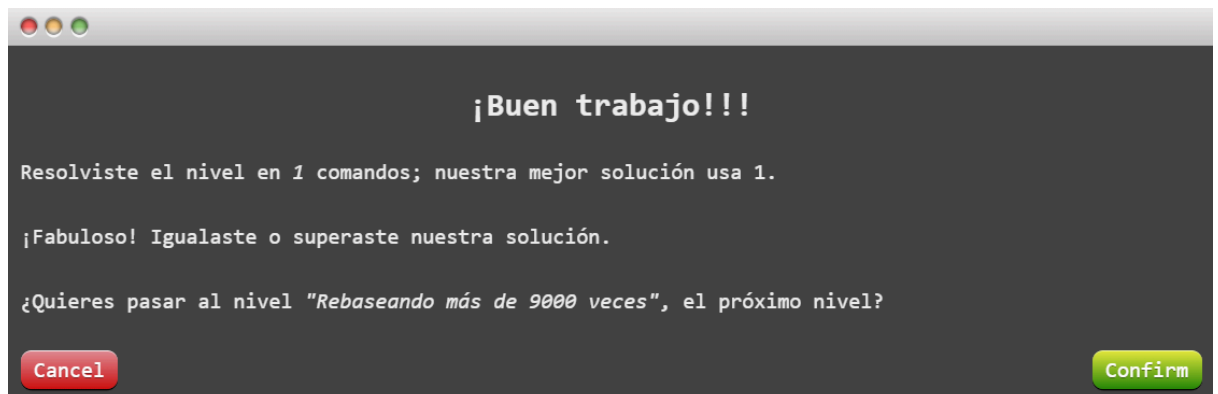
En este ejercicio comprendí que existen diferentes formas de reorganizar y modificar los commits de un proyecto. Entendí que, aunque `rebase -i` es una herramienta útil para cambiar el orden de los commits y editarlos, también puede hacer el proceso más complejo cuando hay muchos cambios. Además, aprendí que `git cherry-pick` es una alternativa que permite seleccionar y mover únicamente los commits necesarios, facilitando la organización del historial y reduciendo la posibilidad de generar conflictos.

16.



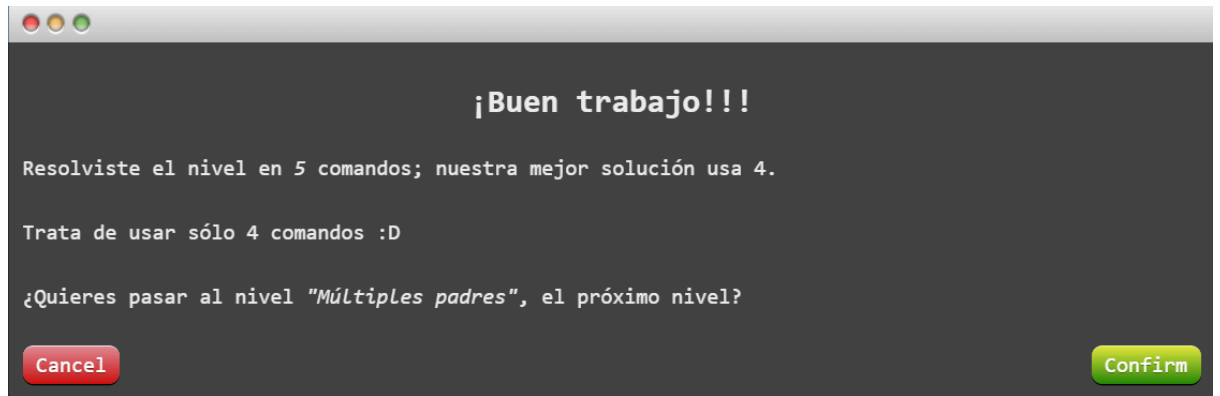
En este ejercicio comprendí que las etiquetas (tags) sirven para marcar de forma permanente un punto importante en el historial de un proyecto. A diferencia de las ramas, que cambian de posición a medida que se realizan nuevos commits, las etiquetas permanecen en el mismo lugar y permiten identificar versiones o momentos relevantes del desarrollo. Entendí que son una herramienta útil para organizar el proyecto y facilitar la referencia a versiones importantes en el futuro.

17.



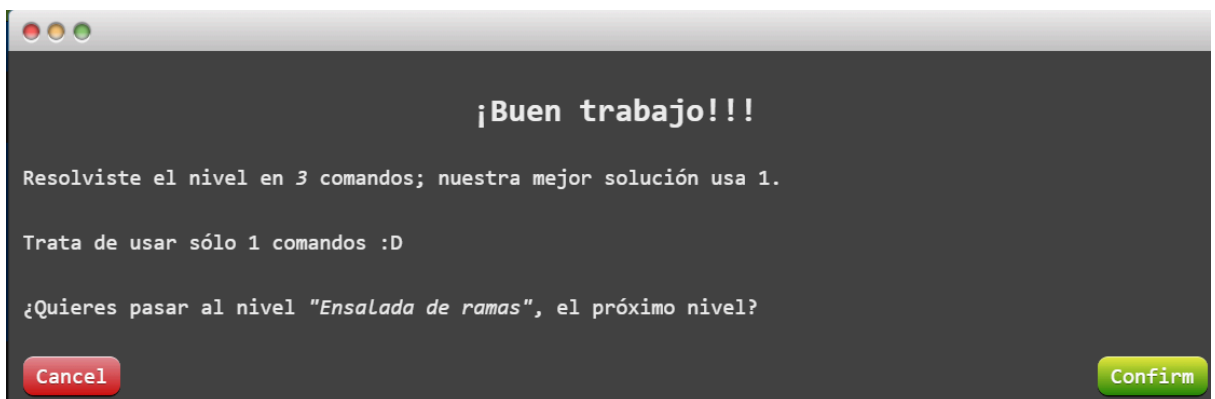
En este ejercicio comprendí que el comando `git describe` permite identificar de manera más sencilla la ubicación actual dentro del historial del proyecto tomando como referencia la etiqueta (tag) más cercana. Entendí que esta herramienta resulta útil para saber en qué punto del desarrollo se está trabajando, especialmente cuando se han realizado varios cambios o se ha navegado entre diferentes commits. De esta manera, es más fácil orientarse y reconocer la relación entre el commit actual y las versiones importantes del proyecto.

18.



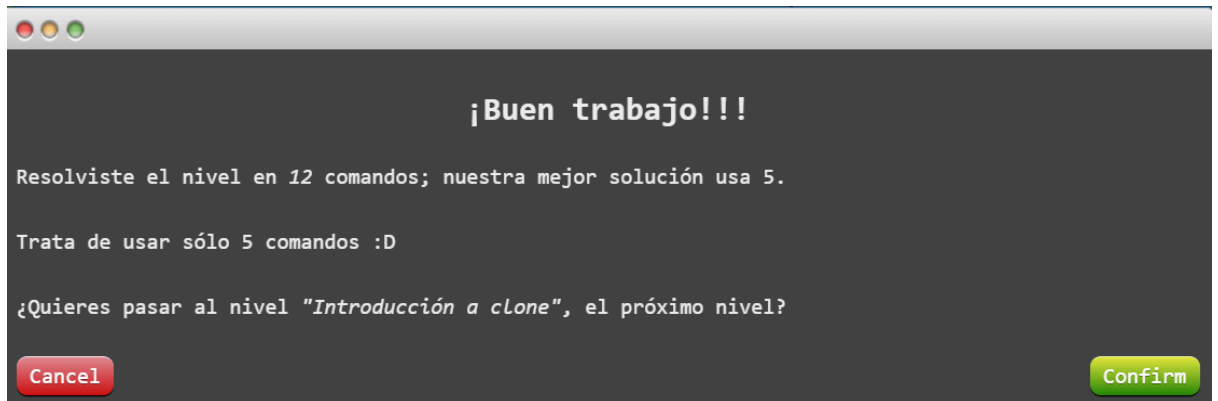
En este ejercicio comprendí que es posible aplicar rebase a varias ramas para reorganizar el historial del proyecto y mantener los commits en un orden secuencial. También aprendí que Git permite realizar este proceso de forma más rápida indicando la rama sobre la que se hará el rebase y la rama que se desea mover en un mismo comando, evitando tener que cambiar de rama manualmente. Esto ayuda a mantener un historial más limpio, ordenado y fácil de seguir cuando se trabaja con varias ramas al mismo tiempo.

19.



En este ejercicio comprendí que, cuando un **commit de fusión (merge)** tiene más de un padre, Git permite indicar cuál de ellos se desea seguir mediante un modificador específico. Entendí que esto facilita la navegación dentro del historial del proyecto cuando existen varias rutas posibles, ya que es posible elegir de forma precisa el padre que se quiere tomar como referencia. De esta manera, se tiene un mayor control al recorrer el historial de commits y trabajar con proyectos que incluyen fusiones de ramas.

20.



En este ejercicio comprendí que es posible actualizar varias ramas de forma independiente para que contengan los cambios más recientes de la rama principal, sin que todas deban modificarse de la misma manera. Entendí que algunas ramas pueden requerir únicamente reorganizar sus commits, mientras que otras necesitan agregar o eliminar cambios antes de quedar listas. Este ejercicio me permitió comprender mejor cómo adaptar el historial de cada rama según las necesidades del proyecto y mantener un historial más ordenado y coherente.

Resultado Final

Selecciona un nivel

PrincipalRemota

Secuencia introductoria

Una breve introducción a la mayoría de los comandos de git

1: Introducción a Los commits de Git

Acelerando

La próxima ración de git. Espero que estés hambriento

1: Desacopla tu HEAD

Moviendo el trabajo por ahí

Ponte cómodo cuando modifiques el directorio fuente

1: Introducción a cherry-pick

Un poco de todo

Un batiburrillo de técnicas, trucos y sugerencias sobre Git

3: Haciendo malabares con Los commits #2

Temas avanzados

¡Para los verdaderos valientes!

1: Rebaseando más de 9000 veces